

Introduction to Scikit-learn & Dataset Handling

Learning Objectives

By the end of this lesson, students will be able to:

- Understand what Scikit-learn is and why it is used.
 - Install and import Scikit-learn.
 - Load datasets from CSV files.
 - Understand features and target variables.
 - Explore datasets using Pandas.
 - Handle missing values.
 - Split data into training and testing sets.
 - Understand why train-test splitting is important.
 - Prepare a dataset for machine learning.
-

What is Scikit-learn?

Scikit-learn (also called **sklearn**) is one of the most popular Machine Learning libraries in Python. It provides simple and efficient tools for building, training, and evaluating machine learning models.

It is built on top of:

- NumPy
- Pandas
- Matplotlib
- SciPy

Scikit-learn allows developers and data scientists to build machine learning models with only a few lines of code.

Why Do We Need Scikit-learn?

Imagine you want to build a model that predicts house prices.

Without Scikit-learn, you would need to:

- Write mathematical formulas yourself.
- Implement optimization algorithms.
- Handle matrix operations manually.
- Calculate model accuracy.

This process is difficult and time-consuming.

Scikit-learn provides ready-made implementations of machine learning algorithms so you can focus on solving real-world problems instead of implementing algorithms from scratch.

Features of Scikit-learn

Scikit-learn offers many useful features:

- Easy to learn and use
 - Open-source and free
 - Supports supervised and unsupervised learning
 - Built-in datasets
 - Data preprocessing tools
 - Feature scaling
 - Train-test splitting
 - Model evaluation
 - Cross-validation
 - Hyperparameter tuning
 - Visualization support with Matplotlib
-

Installing Scikit-learn

Install Scikit-learn using pip:

```
pip install scikit-learn
```

Or in Google Colab:

```
!pip install scikit-learn
```

Usually, Google Colab already includes Scikit-learn.

Importing Required Libraries

Most machine learning projects begin by importing the necessary libraries.

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.model_selection import train_test_split
```

For Linear Regression:

```
from sklearn.linear_model import LinearRegression
```

For Logistic Regression:

```
from sklearn.linear_model import LogisticRegression
```

Machine Learning Workflow

Every machine learning project follows a common workflow.

Collect Data



Load Dataset



Understand Dataset



Clean Dataset



Split Dataset



Train Model



Test Model



Evaluate Performance



Make Predictions

Understanding this workflow is more important than memorizing code.

What is a Dataset?

A **dataset** is a collection of related data organized in rows and columns.

Example:

Hours	Attendance	Previous Marks	Final Marks
2	80	45	52
4	85	60	68
6	90	75	82
8	95	88	94

Each row represents one student.

Each column represents one feature or variable.

Types of Data

Datasets can contain different types of data.

Numerical Data

Contains numbers.

Examples:

- Age
- Salary
- Height
- Weight
- Marks

Example:

18

22

35

50

Categorical Data

Contains labels or categories.

Examples:

Male

Female

Pass

Fail

Yes

No

Text Data

Examples:

- Product Review
- Feedback
- Email
- News Article

Example:

"This product is excellent."

Date and Time Data

Examples:

2026-07-14

14/07/2026

Loading a Dataset

Most datasets are stored as CSV files.

Example:

```
import pandas as pd
```

```
data = pd.read_csv("student_marks.csv")
```

Display the first five rows:

```
data.head()
```

Output:

Hours	Attendance	Previous Marks	Final Marks
2	80	45	52
4	85	60	68
6	90	75	82
8	95	88	94

Understanding the Dataset

Before training any model, always explore the dataset.

View the First Rows

```
data.head()
```

View the Last Rows

```
data.tail()
```

Check Dataset Shape

```
data.shape
```

Output:

(100, 4)

Meaning:

- 100 rows
 - 4 columns
-

View Column Names

`data.columns`

Output:

Hours

Attendance

Previous Marks

Final Marks

Dataset Information

`data.info()`

Shows:

- Number of rows
 - Number of columns
 - Data types
 - Missing values
-

Statistical Summary

`data.describe()`

Provides:

- Mean
- Minimum

- Maximum
 - Standard Deviation
 - Quartiles
-

Missing Values

Sometimes datasets have incomplete information.

Example:

Hours	Attendance	Marks
2	80	55
5	NaN	70
8	95	90

NaN means **Not a Number**, indicating a missing value.

Finding Missing Values

```
data.isnull().sum()
```

Example Output:

Hours	0
Attendance	2
Marks	1

Handling Missing Values

Remove Rows with Missing Values

```
data.dropna()
```

Fill Missing Values

Fill with the column mean:

```
data.fillna(data.mean())
```

Or fill a specific column:

```
data["Attendance"].fillna(85)
```

Features and Target Variable

Machine learning models learn from **features** to predict a **target**.

Example:

Hours	Attendance	Previous Marks	Final Marks
-------	------------	----------------	-------------

Features:

- Hours
- Attendance
- Previous Marks

Target:

- Final Marks
-

Selecting Features

```
X = data[["Hours", "Attendance", "Previous Marks"]]
```

Selecting Target

```
y = data["Final Marks"]
```

Why Separate X and y?

Machine learning models require:

- **X** → Input data (features)
- **y** → Output data (target)

The model learns the relationship between X and y.

X → Machine Learning Model → Prediction

Hours

Attendance

Previous Marks

↓

Predicted Marks

Training and Testing Data

A machine learning model should not be tested using the same data it learned from.

Instead, divide the dataset into two parts:

Training Data

Used to teach the model.

Usually 70–80% of the data.

Testing Data

Used to evaluate the model.

Usually 20–30% of the data.

Example:

100 Students

80 Students → Training

20 Students → Testing

Why Train-Test Split?

If a model is evaluated using the same data it was trained on, it may simply memorize the answers instead of learning patterns.

Using separate training and testing data helps determine how well the model performs on unseen data.

Splitting the Dataset

```
from sklearn.model_selection import train_test_split
```

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
X,  
y,  
test_size=0.2,  
random_state=42  
)
```

Understanding Parameters

test_size

Specifies the proportion of data reserved for testing.

0.2 = 20%

0.3 = 30%

random_state

Controls how the data is shuffled before splitting.

Using the same value (for example, **42**) ensures the split is reproducible each time the code runs.

Output After Splitting

Suppose the dataset has 100 records.

X_train : 80

X_test : 20

y_train : 80

y_test : 20

Built-in Datasets in Scikit-learn

Scikit-learn includes several sample datasets.

Example:

```
from sklearn.datasets import load_iris
```

```
iris = load_iris()
```

Popular built-in datasets include:

- Iris Dataset
- Wine Dataset
- Breast Cancer Dataset
- Diabetes Dataset
- Digits Dataset

These datasets are useful for practicing machine learning without downloading external files.

Common Beginner Mistakes

- Forgetting to split the dataset.
- Training and testing on the same data.
- Choosing the wrong target column.
- Ignoring missing values.
- Using text data without preprocessing.
- Forgetting to import required libraries.
- Not checking the dataset before training.

Best Practices

- Understand the dataset before building a model.
 - Check for missing values.
 - Separate features and target correctly.
 - Always use a train-test split.
 - Keep your code organized.
 - Start with small datasets before working on larger ones.
 - Use meaningful variable names such as `X_train`, `X_test`, `y_train`, and `y_test`.
-

Summary

In this lesson, you learned:

- What Scikit-learn is and why it is widely used.
- How to install and import Scikit-learn.
- How to load datasets using Pandas.
- How to inspect and understand a dataset.
- How to identify and handle missing values.
- The difference between features (X) and target (y).
- Why train-test splitting is essential.
- How to split data using `train_test_split()`.
- The standard workflow for preparing data before training a machine learning model.

This forms the foundation for all upcoming machine learning algorithms, including **Linear Regression**, **Logistic Regression**, **Decision Trees**, and many others. Mastering dataset handling ensures that students can confidently prepare data for any machine learning project.